

REST Assured API Response Validation

Five practical approaches for Senior SDET / Test Automation Lead interviews

What this guide covers

JSONPath extraction, direct body() assertions, POJO deserialization, JSON Schema validation, and Java Stream-based business validation.

Sample API response

```
{
  "id": 101,
  "name": "Swapnil",
  "address": {
    "city": "Pune",
    "state": "Maharashtra"
  },
  "orders": [
    { "orderId": 501, "product": "Laptop", "price": 75000 },
    { "orderId": 502, "product": "Mobile", "price": 25000 }
  ]
}
```

1. JSONPath validation — extract values and assert separately

Use REST Assured's `jsonPath()` to read values from the response. Then use TestNG or Java assertions.

```
Response response = given()
    .when()
    .get("/api/customer/101");

int id = response.jsonPath().getInt("id");
String name = response.jsonPath().getString("name");

Assert.assertEquals(id, 101);
Assert.assertEquals(name, "Swapnil");
```

Use when: You need to extract selected fields and perform separate or conditional assertions.

2. Direct body() validation — REST Assured + Hamcrest

Use `body()` in the REST Assured validation chain. JSONPath expressions identify the fields; Hamcrest matchers compare their values.

```
given()
    .when()
    .get("/api/customer/101")
    .then()
    .statusCode(200)
    .body("id", equalTo(101))
    .body("name", equalTo("Swapnil"))
```

```
.body("address.city", equalTo("Pune"));
```

Use when: You want concise, readable assertions directly in the API test.

3. POJO validation — Jackson deserialization

Convert the JSON response into Java objects, then validate using getters. Create matching Java model classes such as Customer, Address and Order.

```
Response response = given()  
    .when()  
    .get("/api/customer/101");  
  
Customer customer = response.as(Customer.class);  
  
Assert.assertEquals(customer.getId(), 101);  
Assert.assertEquals(customer.getName(), "Swapnil");  
Assert.assertEquals(customer.getAddress().getCity(), "Pune");
```

Use when: The response is complex or deeply nested, or you want reusable models and business-level assertions.

4. JSON Schema validation — validate the API contract

Validate the response structure against a predefined JSON Schema. The schema can specify required fields, data types, nested properties and array item structure.

```
given()
  .when()
    .get("/api/customer/101")
  .then()
    .statusCode(200)
    .body(matchesJsonSchemaInClasspath("customer-schema.json"));
```

Use when: You need to check contract or structural compliance, especially across many API tests.

5. Java Streams — validate collections and business rules

After deserializing into POJOs, use Java Streams for validations across dynamic arrays without hardcoding array indexes.

```
Customer customer = response.as(Customer.class);

boolean allPricesValid = customer.getOrders()
  .stream()
  .allMatch(order -> order.getPrice() > 0);

Assert.assertTrue(allPricesValid);
```

Use when: You need to validate every item in a collection or enforce business rules such as positive prices, valid statuses or totals.

Quick comparison

Approach	Main purpose	Typical tools
JSONPath	Extract individual JSON values	REST Assured jsonPath()
body()	Direct response assertions	REST Assured + Hamcrest
POJO	Map JSON to Java objects	Jackson
JSON Schema	Validate response structure / contract	JSON Schema Validator
Java Streams	Validate collections and business rules	Java + TestNG/JUnit

Important interview distinction

These are related but not all independent technologies: `body()` is a REST Assured method; the expression such as `address.city` is a JSONPath expression; `equalTo()` is a Hamcrest matcher. POJO mapping is commonly handled by Jackson. JSON Schema validation checks the response contract.

Interview-ready answer

“I generally use five approaches for API response validation: JSONPath extraction, direct body assertions using REST Assured and Hamcrest, POJO deserialization using Jackson, JSON Schema validation, and Java-based validation using Streams and assertions. For simple field checks, I use JSONPath with body assertions. For complex nested responses, I prefer POJOs. For contract validation, I use JSON Schema. For dynamic collections and business rules, I use Java Streams. The choice depends on response complexity and the validation objective.”

Note: The examples assume the required REST Assured, Hamcrest, Jackson, JSON Schema Validator and TestNG dependencies are available in your Maven project. The Customer, Address and Order POJO classes and customer-schema.json must be created to match the sample response.